



Evaluating LLMs for Automatic Hint Generation in Programming

An Experimental Study on Infoarena and HackerRank programming problems

Motivation & Research Objective



Research Goal

Evaluate LLMs for generating useful programming hints

Systematically assess how Large Language Models perform in automated tutoring scenarios, analyzing key factors that drive pedagogical value, clarity, and overall accuracy.

Key Questions

- **Utility:** How useful are the generated hints in practice?
- **Model Excel:** Which models and scenarios produce the best results?
- **Context Impact:** What specific context improves hint quality?
- **Failure Modes:** Where do LLMs fail, and how can they be improved?

Evaluation Pipeline & Hint Generation Flow



Data Processing & Curation Pipeline



01. CLEANING

Input Cleaning

Removed incomplete, invalid and malformed submissions.

02. DEDUPLICATION

Redundancy Removal

Eliminated identical or near-duplicate solutions.

03. SIMILARITY

Similarity Analysis

Detected structurally similar code using JPlag.

04. FILTERING

Execution Filtering

Separated valid runs from compile/runtime errors and timeouts.

05. METADATA

Metadata Processing

Standardized problem, language and submission information.

06. FINAL DATASET

Final Selection

Retained representative solutions for profiling and hint generation.

Tools: scraping scripts execution runner JPlag custom preprocessing / profiling scripts, cppcheck for redundant code analysis

Dataset Organization & Statistics — InfoArena + HackerRank

Structure and quantitative overview of two competitive-programming submission datasets.

Total Problems

72 Combined

Total Submissions

129,454

Test Executions

278,931

Languages & Users

Multi-Language support

C++, Java, Python, Pascal

Common Dataset Folder Structure

Shared archive-style organization used across the datasets

 <problem>/	— dataset folder for one problem
 <problem>.json	— metadata, limits, statement, I/O
 statement.txt	— full problem statement
 solution.txt	— editorial solution, when available
 tests/	— input/output test files
 <problem>.csv	— one row per submission
 submissions/	— original source code
 solutions/	— extracted source files
 c_sol/ cpp_sol/ java_sol/	— code grouped by language
 details/	— per-test evaluation results
 details_local/	— local re-evaluation results

InfoArena Dataset Breakdown

- **66** complex competitive problems
- **124,716** indexed active submissions
- **160,130** full submission metadata records
- **217,313** test-level outcome files mapped

HackerRank Dataset Breakdown

- **6** complex competitive problems
- **4,738** raw scraped code submissions
- **3,223** cleaned + deduplicated records
- **61,618** test-level execution logs processed

Algorithm Profiling Approaches

Direct Classification vs. Text-Level Assisted Classification

Research Question: Does generating an intermediate text-level description improve algorithm identification?

APPROACH 1

Direct Classification

Direct route: Model predicts algorithm directly from source code



- Input: Complete raw programming submission
- Output: Single classification target representing the main algorithm

APPROACH 2

Text-Level Assisted Classification

Text-level assisted route: Generates explanation prior to identification



- Step 1: Model generates a natural-language description of the solution
- Step 2: Description is processed to identify the underlying algorithm

INFOARENA DATASET (MAXFLOW)

Model: **Qwen 3.5 9B**

Gold Labels: **DeepSeek-v4 Pro** (100% accuracy on 50 manually verified samples)

Evaluation Scenarios:

- Text-Level Assisted: **97.4% accuracy**

HACKERRANK DATASET

Model: **GPT-OSS 20B**

Gold Set: **76 manually curated samples** (4 problems)

Evaluation: **1,354 submissions** (2 problems)

Taxonomy Agreement: **93.5% General** | **82.1% Detailed**

Hint Generation Setup — Prompt Inputs



We compare what information is given to the model before generating a Socratic hint.

1. Input Conditions

V2 — No Tests

- Problem statement
- Student source code

V7 — With Private Tests

- Problem statement
- Student source code
- Private failed-test evidence

** Private test evidence is used only by the model, not shown to the student.*

2. Models

Evaluate the same submissions with two distinct LLM architectures:

GPT-OSS-20B

Qwen3.5-9B

Same Task:

Generate one short Socratic hint.

3. Prompt Constraints

System Prompts strictly enforce:

"Do not reveal the exact bug, fix, failing test, edge case, algorithm, or code location."

"Do not mention tests, test numbers, expected outputs, or observed outputs."

"Return only the hint."

4. Evaluation Data

SUBMISSION-LEVEL

- 3 problems | 40-70 submissions per problem
- Diverse score levels & algorithmic approaches
- One problem focuses specifically on TLE cases

TRACE-LEVEL

- 100 diverse solution traces
- Initial attempt → improved attempt
- Used to analyze how solutions & errors evolve

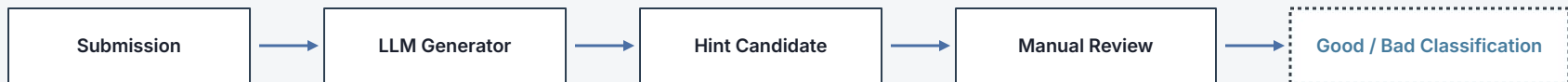
Experimental Takeaway

The experiment isolates whether private execution evidence improves hint quality or increases information leakage.



Evaluation Pipeline & Hint Quality Criteria

System Architecture & Review Workflow



✓ Good Hint Taxonomy

- Useful and contextually relevant to the specific error
- Guides student conceptually toward discovery
- Strictly avoids offering direct or exact code fixes

✗ Bad Hint Taxonomy

- Reveals the entire solution algorithm upfront
- Provides exact fix, line deletions, or copy-paste code
- Misleading or architecturally incorrect advice

Infoarena Example

"The problem asks for a contiguous subarray, which implies a relationship between the current prefix XOR and a previous prefix XOR."

HackerRank Example (Leaked Fix)

"Have you ensured that the node you enqueue at the start is also marked as visited before the loop begins?"

Model Evaluation — GPT-OSS-20B vs Qwen3.5-9B — HackerRank

Hint quality across two models and two input conditions, evaluated on 89 comparable submissions.

Main idea: We evaluate whether giving the model private failed-test evidence improves hints or makes them too revealing.

V2: Statement + Code

GPT-OSS
20B

GOOD Rate

74.16%

66 GOOD / 23 BAD

Failure Types

- Leak: 0 | Bug: 20 | Alg: 0
- Impl: 6 | Fix: 17 | Vague: 8

V7: Statement + Code + Failed-Tests

GOOD Rate

61.80%

55 GOOD / 34 BAD

Failure Types

- Leak: 2 | Bug: 40 | Alg: 11
- Impl: 11 | Fix: 23 | Vague: 1

V2 vs V7 Effect

GPT-OSS: V7 worsens quality (74.16% → 61.80%)

Qwen: V7 improves quality (37.08% → 46.07%)

Private tests help Qwen target better, but increase leakage risk.

GOOD Rate

37.08%

33 GOOD / 56 BAD

Failure Types

- Leak: 0 | Bug: 54 | Alg: 14
- Impl: 23 | Fix: 6 | Vague: 2

Qwen3.5
9B

GOOD Rate

46.07%

41 GOOD / 48 BAD

Failure Types

- Leak: 18 | Bug: 41 | Alg: 12
- Impl: 16 | Fix: 5 | Vague: 6

Model Comparison

Best: GPT-OSS V2 (74.16% GOOD)

Worst: Qwen V2 (37.08% GOOD)

Qwen V7 improves over Qwen V2 but remains below GPT-OSS.

GPT-OSS is more stable overall.

Methodology Note

OK/BAD verdicts are based on manual review. BAD marks hints that reveal private evidence, expose the exact bug/fix, become too implementation-specific, or are too vague to be useful.

Head-to-Head Comparison & Annotator Agreement

Pairwise comparison of model performance and human annotation consistency

Scenario Performance

	V2 No Tests	V7 With Tests
GPT-OSS	74.16%	61.80%
Qwen3.5	37.08%	46.07%

Scenario Changes (V2 → V7)

GPT-OSS: **-12.36 pp**
Qwen3.5: **+8.99 pp**

Head-to-Head Outcomes

V2 — NO TESTS

Both Good: 26

GPT Wins: 40

Qwen Wins: 7

Both Bad: 16

V7 — WITH PRIVATE TESTS

Both Good: 28

GPT Wins: 27

Qwen Wins: 13

Both Bad: 21

Overall Head-to-Head Win Rate

GPT-OSS: **77.01%**
Qwen3.5: **22.99%**

Inter-Annotator Agreement

Cohen's Kappa

$\kappa = 0.68$

Raw Agreement: **84.44%**

Agreement Cases: **76**

Disagreement Cases: **14**

Double-Annotated Examples: **90**

Note: $\kappa = (P_o - P_e) / (1 - P_e)$, where $P_o = 76/90 = 84.44\%$ and P_e is computed from marginal OK/BAD distributions.

Observation: Most disagreements came from judging whether a hint revealed the exact bug or provided a concrete fix.

Methodology & Scope Note

Head-to-head results compare GPT-OSS and Qwen on identical submissions. Cohen's Kappa measures agreement between the two human annotators, Emilia and Alex, and is independent of model performance.

Trace-Level Hints — FIRST vs. UPGRADE

100 paired solution traces

Transition Outcomes

Improved

32

BAD → OK

Regressed

31

OK → BAD

Stable Good

17

OK → OK

Still Problematic

20

BAD → BAD

WHAT IMPROVED

- Exact bug leakage: **45 → 22**
- Too vague hints: **7 → 0**

WHAT GOT WORSE

- Strategy revealed: **2 → 21**
- Concrete fix given: **1 → 32**

Key Conclusion & Takeaway

Using the improved attempt helped avoid vague or overly diagnostic hints, but often pushed the model toward strategy or code leaks. The upgrade changed the failure mode more than the overall success rate.

LLM Debugging: Core Capabilities & Limits



Analyzing where modern language models excel and struggle in automated code feedback

✓ Strengths

- Recognizing **common algorithmic patterns**
- Providing **conceptual hints** instead of code
- Generating **clear, concise explanations**

✗ Limitations

- High risk of **code spoilers** (leakage)
- Struggles with **ad hoc problems**
- Struggles with **vague submissions**
- Difficulty maintaining **optimal specificity**

↑ How to Improve

- **Richer test evidence**
official outputs + better metadata
- **Failure-aware prompting**
separate handling for timeout / WA / RE
- **Better support for partials**
gradual hints, not immediate rejection

Key Takeaway

"LLMs are promising for educational debugging, but better results require richer execution evidence, safer prompting, and more adaptive hints for partial or ambiguous solutions."